Lecture 10. Functions

Writing functions, conditional execution, arguments, return values, environments

Contents

10.1 When to write a function **2**

10.2 Functions are for humans and computers **2**

10.3 Conditional execution **3**

10.3.1 Conditions . . . 3

10.3.2 Multiple conditions . . . 3

10.3.3 Code style . . . 4

10.4 Function arguments **4**

10.4.1 Checking values . . . 4

10.4.2 Dot-dot-dot . . . 5

10.5 Return values **5**

10.6 Environment **6**

10.7 Selected exercises **6**

10.1 When to write a function

Motivation. Copy-and-paste is the most common source of bugs in analysis code: one copy gets updated and another does not. Functions remove the copies. They also give a name to an idea, which makes code easier to read and to change.

Consider rescaling each column of a data frame to $[0, 1]$:

```
df <- tibble::tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
df$a <- (df$a - min(df$a, na.rm = TRUE)) /
  (max(df$a, na.rm = TRUE) - min(df$a, na.rm = TRUE))
df$b <- (df$b - min(df$b, na.rm = TRUE)) /
  (max(df$b, na.rm = TRUE) - min(df$a, na.rm = TRUE))
# ... and the same for c and d
```

There is a copy-paste error in the `b` line (`min(df$a)`) that is easy to miss. A rule of thumb: write a function whenever you have copied a block of code more than twice (DRY, "do not repeat yourself").

How to write one. First analyse the code: how many inputs does it have? Here one, so rename it to a temporary general name, `x`. Then remove duplication: the code computes the range three times, and `range()` does it once:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(c(-10, 0, 10))
```

```
## [1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
## [1] 0.00 0.25 0.50   NA 1.00
```

Three steps create a function: pick a **name**, list the **arguments** inside `function()`, and put the code in the **body** in braces. It is easier to start with working code and turn it into a function than the reverse. The four columns now take one line each, `df$a <- rescale01(df$a)`, and Lecture 12 removes even that repetition.

A further advantage: requirements change in one place. If some values are infinite, the range is infinite and everything becomes `NaN`; one change to the function fixes every use:

```
x <- c(1:10, Inf)
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(x)
```

```
##  [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667 0.7777778
##  [9] 0.8888889 1.0000000       Inf
```

10.2 Functions are for humans and computers

R does not care about names or comments, but readers do.

- **Names** should be short but clearly describe what the function does: verbs for functions (`impute_missing()`, `collapse_years()`), nouns for arguments. Avoid both too short (`f()`) and undescriptive names (`my_awesome_function()`).
- Use one naming style consistently, `snake_case` or `camelCase`, never a mix like `col_mins()` and

`rowMaxes()`. A common prefix groups a family of functions for autocompletion: `input_select()`, `input_checkbox()`, `input_text()`.

- Never overwrite existing functions and variables: `T <- FALSE`, `c <- 10`, `mean <- function(x) sum(x)` lead to baffling bugs.
- **Comments** explain the *why*, not the what or how, which the code itself should make clear. Lines of dashes such as `# Load data ---` break a script into readable sections (RStudio can jump between them).

10.3 Conditional execution

Motivation. A function often has to behave differently for different inputs: handle a special case, check a condition, choose between methods. if–else statements express these branches.

```
if (condition) {
  # code executed when condition is TRUE
} else {
  # code executed when condition is FALSE
}
```

```
has_name <- function(x) {
  nms <- names(x)
  if (is.null(nms)) {
    rep(FALSE, length(x))
  } else {
    !is.na(nms) & nms != ""
  }
}
has_name(c(a = 1, 2, b = 3))
```

```
## [1]  TRUE FALSE  TRUE
```

This uses the convention that a function returns the last value it computed.

10.3.1 Conditions

The condition must be a single `TRUE` or `FALSE`; a longer logical vector is an error (it used to be a warning), and `NA` is an error:

```
if (c(TRUE, FALSE)) {}
```

```
## Error in if (c(TRUE, FALSE)) {: the condition has length > 1
```

```
if (NA) {}
```

```
## Error in if (NA) {: missing value where TRUE/FALSE needed
```

Use `||` and `&&` (which look at one value and stop as soon as the answer is known) to combine conditions, never the vectorised `|` and `&` inside `if`. If you have a logical vector, summarise it with `any()` or `all()`. Be careful with `==`: it is vectorised and returns `NA` for missing values. `identical()` is strict, returning one `TRUE` or `FALSE` but also distinguishing types (`identical(0L, 0)` is `FALSE`); for real numbers use `dplyr::near()` (Lecture 3), since `sqrt(2)^2 == 2` is `FALSE`.

10.3.2 Multiple conditions

Chain `if (this) {...} else if (that) {...} else {...}`. With many alternatives, `switch()` selects code by name or position:

```
calc <- function(x, y, op) {
  switch(op,
    plus = x + y,
    minus = x - y,
    times = x * y,
    divide = x / y,
    stop("Unknown op!")
  )
}
calc(6, 3, "divide")
```

```
## [1] 2
```

`cut()` discretises continuous variables, replacing long `if` chains.

10.3.3 Code style

An opening brace is never on its own line and is always followed by a new line; a closing brace always goes on its own line, unless followed by `else`. Indent the contents by two spaces. A very short `if` can sit on one line: `x <- if (y < 20) "Too low" else "Too high"`. Putting `else` on a new line after a closing brace is a syntax error at the top level, because R considers the `if` statement complete.

10.4 Function arguments

Motivation. Arguments are the interface of a function. A good interface separates what the data are from how to compute, gives sensible defaults, and fails loudly on invalid input.

Arguments fall into two sets: those supplying the **data** to compute on, and those controlling the **details** of the computation. Data arguments come first; detail arguments come last and usually have **default values**:

```
mean_ci <- function(x, conf = 0.95) {
  se <- sd(x) / sqrt(length(x))
  alpha <- 1 - conf
  mean(x) + se * qnorm(c(alpha / 2, 1 - alpha / 2))
}
x <- runif(100)
mean_ci(x)
```

```
## [1] 0.4194814 0.5276171
```

```
mean_ci(x, conf = 0.99)
```

```
## [1] 0.4024921 0.5446064
```

The default should almost always be the most common value (an exception is `na.rm = FALSE`: missing values are important, so ignoring them should be a deliberate choice). When calling, omit the names of data arguments and name detail arguments in full: `mean(1:10, na.rm = TRUE)`, not `mean(, TRUE, x = c(1:10, NA))`. Put spaces around `=` in calls and after commas.

Common argument names: `x`, `y`, `z` (vectors), `w` (weights), `df` (data frame), `i`, `j` (indices), `n` (length or number of rows), `p` (number of columns). Follow existing conventions such as `na.rm`.

10.4.1 Checking values

Recycling rules make some mistakes silent:

```
wt_mean <- function(x, w) sum(x * w) / sum(w)
wt_mean(1:6, 1:3)
```

```
## [1] 7.666667
```

The weights were recycled to length 6 without any warning (the length 6 is a multiple of 3). Check important preconditions and throw an error with `stop()`, or use `stopifnot()`, which checks that each argument is `TRUE`:

```
wt_mean <- function(x, w, na.rm = FALSE) {
  stopifnot(is.logical(na.rm), length(na.rm) == 1)
  stopifnot(length(x) == length(w))
  if (na.rm) {
    miss <- is.na(x) | is.na(w)
    x <- x[!miss]
    w <- w[!miss]
  }
  sum(w * x) / sum(w)
}
wt_mean(1:6, 6:1, na.rm = "foo")
```

```
## Error in wt_mean(1:6, 6:1, na.rm = "foo"): is.logical(na.rm) is not TRUE
```

Do not overdo it: checks are a trade-off between time spent and robustness.

10.4.2 Dot-dot-dot

Functions such as `sum()` take any number of inputs through the special argument `...`, which captures all unmatched arguments and can be passed on to another function:

```
commas <- function(...) stringr::str_c(..., collapse = ", ")
commas(letters[1:10])
```

```
## [1] "a, b, c, d, e, f, g, h, i, j"
```

The price is that misspelled argument names are swallowed without error: `sum(x, na.mr = TRUE)` adds `TRUE` (that is, 1) to the sum.

10.5 Return values

By default a function returns the last value it evaluates. `return()` returns early, which is useful when the input is a simple special case, or to keep the long branch of an `if` un-indented:

```
complicated_function <- function(x, y, z) {
  if (length(x) == 0 || length(y) == 0) {
    return(0)
  }
  # complicated code here
}
```

Pipeable functions. There are two types: *transformations* return a modified version of their first argument; *side-effect* functions (plotting, printing, saving) should return their first argument *invisibly*, so that it is not printed but can still be piped on:

```
show_missings <- function(df) {
  n <- sum(is.na(df))
  cat("Missing values: ", n, "\n", sep = "")
  invisible(df)
}
mtcars %>%
  show_missings() %>%
```

```
  mutate(mpg = ifelse(mpg < 20, NA, mpg)) %>%
  show_missings()
```

```
## Missing values: 0
## Missing values: 18
```

10.6 Environment

Motivation. A function body can mention variables that are not its arguments. Knowing where R looks for them explains both a common source of bugs and why R functions can be so flexible.

```
f <- function(x) {
  x + y
}
y <- 100
f(10)
```

```
## [1] 110
```

```
y <- 1000
f(10)
```

```
## [1] 1010
```

In many languages `f` would be an error because `y` is not defined inside it. R uses **lexical scoping**: a name not found inside the function is looked up in the environment where the function was *defined*, here the global environment. The result therefore depends on the current value of a global variable, which is a recipe for bugs; avoid writing functions like this deliberately. The same rules let R override even `+` or `(`, which makes the language extremely flexible.

Defined, not called [Supplement]

Lexical scoping looks where the function was created, not where it is called:

```
make_adder <- function(y) function(x) x + y
add2 <- make_adder(2)
y <- 1000
add2(10)
```

```
## [1] 12
```

`add2` remembers the `y = 2` of the environment in which it was created (a *closure*); the global `y` is irrelevant.

10.7 Selected exercises

Motivation. The book's exercises on functions are about interfaces: which inputs should be arguments, how vectorised and scalar conditions differ, and what a default value really is. A last exercise pins down when lexical scoping looks a value up.

Exercise 10.1 (R4DS Exercise 19.2.1.1)

Why is `TRUE` not an argument of `rescale01()`? What would happen if `x` contained a single missing value and `na.rm` were `FALSE`?

```
rescale01_strict <- function(x) {
  rng <- range(x, na.rm = FALSE)
```

```
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01_strict(c(1, 2, NA, 4))
```

```
## [1] NA NA NA NA
```

Solution.

Step 1. Arguments are for what the caller may reasonably want to change. `rescale01()` should map the observed values to $[0, 1]$ whatever the data, and that requires ignoring missing values when computing the range; there is no sensible use with `na.rm = FALSE`, so the value is fixed inside the body rather than exposed.

Step 2. With `na.rm = FALSE`, one `NA` makes `range()` return `c(NA, NA)`. Every subtraction and division then involves `NA`, so the whole result is `NA`, including the entries that were observed, as the output shows. A single missing value would destroy the column. ■

Exercise 10.2 (R4DS Exercises 19.4.4.1 and 19.4.4.5)

(a) What is the difference between `if` and `ifelse()`?
(b) What happens if `switch()` is used with numeric values?

```
ifelse(c(TRUE, FALSE, NA), "yes", "no")
```

```
## [1] "yes" "no"  NA
```

```
if (c(TRUE, FALSE)) "yes" else "no"
```

```
## Error in if (c(TRUE, FALSE)) "yes" else "no": the condition has length > 1
```

```
switch(2, "a", "b", "c")
```

```
## [1] "b"
```

```
is.null(switch(4, "a", "b", "c"))
```

```
## [1] TRUE
```

Solution.

(a) `if` is control flow: it evaluates one condition, which must be a single `TRUE` or `FALSE`, and runs one of two blocks of code. `ifelse(test, yes, no)` is a vectorised function: it returns a vector of the length of `test`, taking each element from `yes` or `no` and giving `NA` where `test` is `NA`. Use `if` inside functions to choose what to do, `ifelse()` inside `mutate()` to compute a column.

(b) With a number n as first argument, `switch()` ignores names and returns the nth of the remaining arguments. If n exceeds their number, it returns `NULL` invisibly, which is why the last call prints nothing unless wrapped, as here in `is.null()`. ■

Exercise 10.3 (R4DS Exercise 19.5.5.3)

The default value of the `method` argument of `cor()` is `c("pearson", "kendall", "spearman")`. What does that mean, and which value is used by default?

```
centre <- function(x, type = c("mean", "median")) {
  type <- match.arg(type)
  switch(type, mean = mean(x), median = median(x))
}
centre(c(1, 2, 10))
```

```
## [1] 4.333333
```

```
centre(c(1, 2, 10), "median")
## [1] 2

centre(c(1, 2, 10), "mode")
## Error in match.arg(type): 'arg' should be one of "mean", "median"
```

Solution. The vector lists the allowed values, and the function body calls `match.arg()`. If the caller leaves the argument out, `match.arg()` receives the whole vector and returns its first element, so `cor()` uses `"pearson"`. If the caller supplies a value, `match.arg()` checks that it is one of the listed values (partial matching is allowed) and signals an error otherwise. The small function above uses the same idiom: the default is the mean, the median can be requested, and an unlisted value is rejected with an informative message. The idiom documents the choices in the function's signature. ■

Exercise 10.4 (when is a free variable looked up)

Predict the value of `f()`, and explain it with lexical scoping.

```
f <- function() {
  x <- 1
  g <- function() x
  x <- 2
  g()
}
f()
## [1] 2
```

Solution [Supplement].

Step 1. `g` has no local `x`, so by lexical scoping `x` is looked up in the environment in which `g` was defined, the execution environment of `f`.

Step 2. Lexical scoping fixes *where* to look, not *when*: the lookup happens when `g()` is called. By then `x` in `f`'s environment has been changed to 2, so `g()` returns 2.

Step 3. This is consistent with the closure in the note above: `add2` sees $y = 2$ because nothing changes the `y` in its defining environment after it is created, while the global `y` lives in a different environment. ■